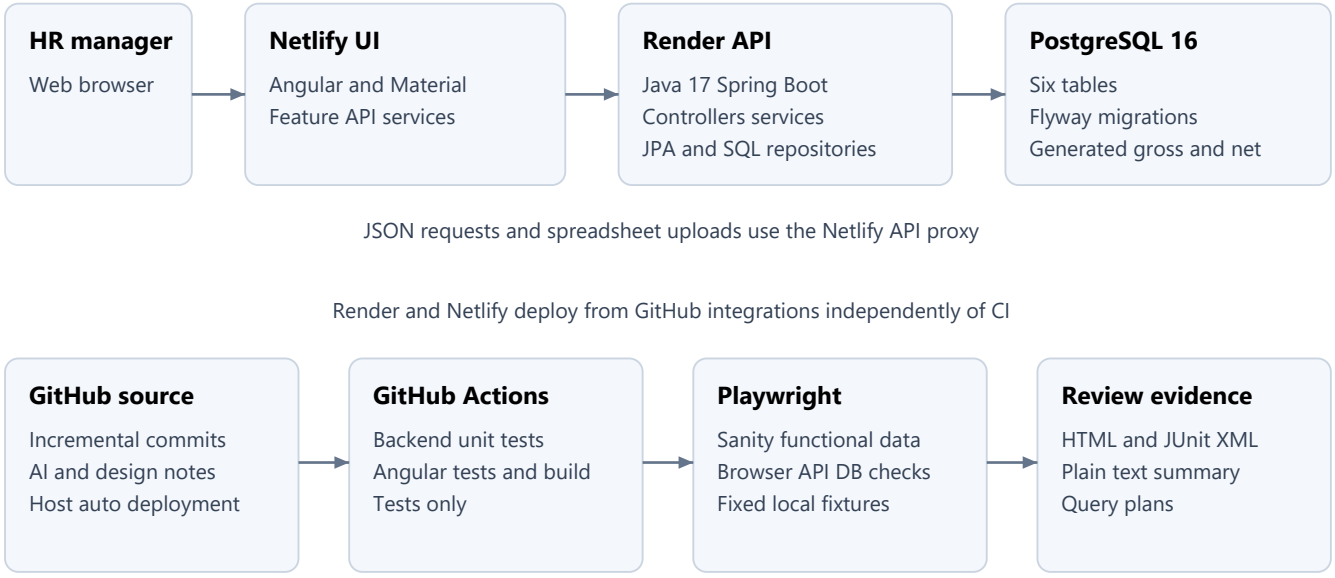
Salary Management Technical Guide

Implementation and review guide for the Incubyte Java and Angular assessment. This guide describes the current application at commit **1dc488e**, its test evidence, and the deployment configuration.

System architecture

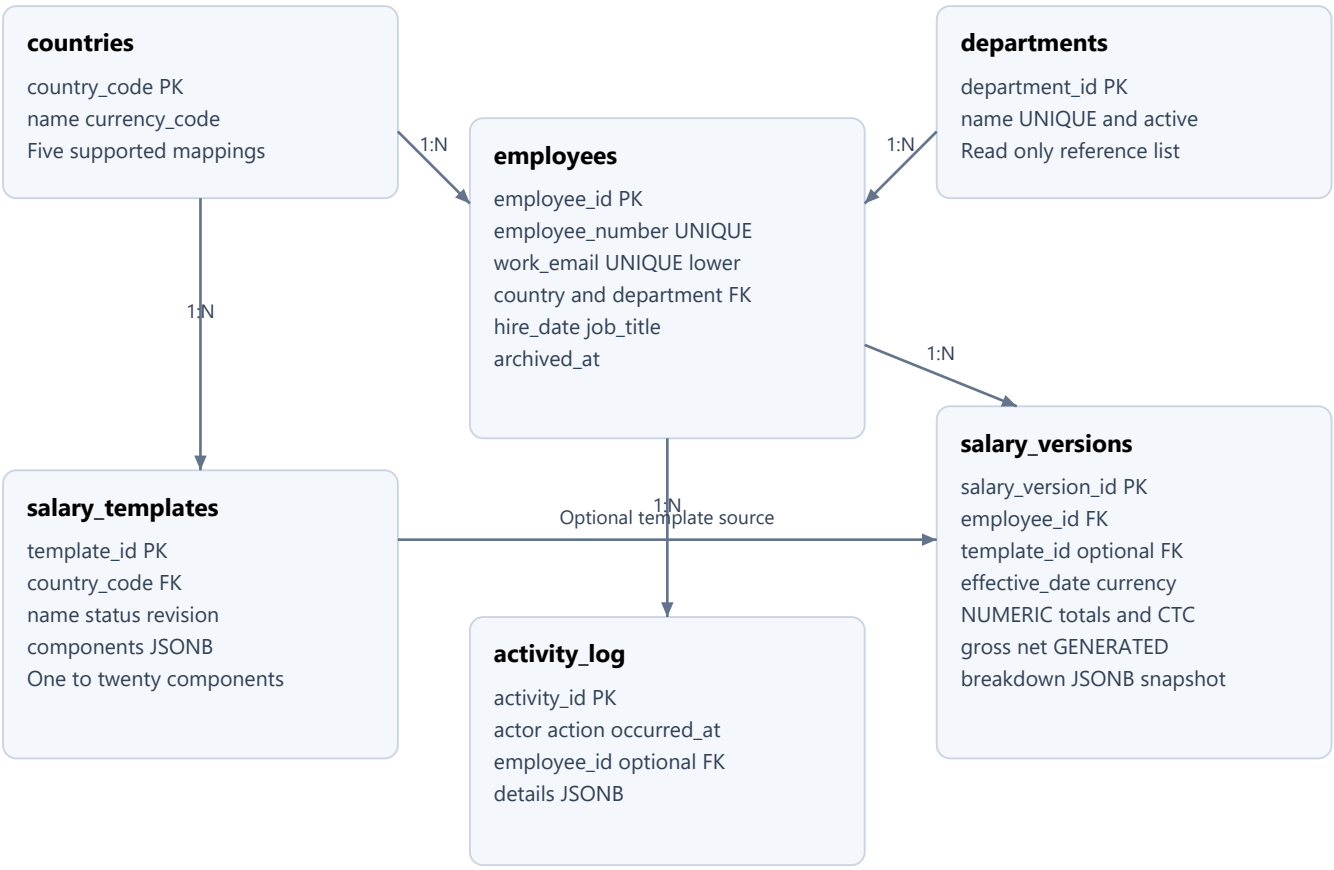


The UI calls typed feature API services. A shared core API service handles HTTP access. Controllers validate DTOs, application services enforce business rules and transaction boundaries, and repositories execute relational queries. Flyway owns schema changes; Hibernate validates the resulting schema.

Layer	Pinned implementation
API	Java 17 and Spring Boot 3.4.2; Spring Data JPA; MapStruct 1.6.3
Database	PostgreSQL 16; local verified server 16.15; Flyway migrations V1 to V3
UI	Angular 22.2.2 and Angular Material 22.2.2; standalone components; strict TypeScript 6.0.3
Checks	JUnit 5 and Mockito; Vitest and Angular TestBed; Playwright 1.52.0 with TypeScript
Hosting	Render Docker API and PostgreSQL; Netlify static Angular app and API proxy

The hosting diagram describes the configured target. The local app and CI are verified; public endpoints still require deployment and smoke checks.

Database model



Six tables cover the chosen scope. Country and department rows are reference data. Templates store a bounded component array in JSONB; salary versions preserve the resolved breakdown and typed money totals. Imports need no batch or row persistence tables.

Storage rules

Employee numbers are unique; work email has a case-insensitive unique index. Required references use foreign keys. Employee archive is a timestamp, not a hard delete. Salary dates are unique per employee, and money columns are NUMERIC(14,2).

```
gross = basic_amount + allowances_amount
net   = basic_amount + allowances_amount - deductions_amount
CHECK (net >= 0)
```

Both totals are PostgreSQL stored generated columns. Net uses base columns because generated expressions cannot reference another generated column. Salary queries read these stored totals.

Indexes and relationship preservation

Active employee ordering uses (archived_at, employee_number), with country and department indexes for filtering. Salary history uses (employee_id, effective_date DESC). Template country/status/name and activity time indexes support their list screens. Foreign keys preserve template provenance, employee salary history, and audit links.

Country and department APIs currently expose reference lists. No department create/edit screen or endpoint is implemented.

Salary calculation and history

HR creates a country-specific template and assigns it to an employee using annual CTC. CTC includes gross earnings plus employer contributions. Net is gross minus employee deductions. All resolved monetary values use BigDecimal with two decimals and HALF_UP rounding.

Component rule	Meaning
Fixed	An annual amount; monthly calculations divide it by twelve
Percentage of CTC	A percentage of the CTC for the period being calculated
Percentage of basic	A percentage of resolved basic pay for that period
Balancing allowance	The remaining CTC after basic, other earnings, and employer contributions

A template has one to twenty uniquely named components, exactly one basic component, and exactly one balancing allowance. Basic is fixed or a percentage of CTC. Percentages cannot exceed 100; balancing value must be zero in the template. Negative amounts, a negative remainder, and deductions above gross are rejected.

Worked example in INR

Annual CTC is 1,200,000.00. Basic is 40 percent of CTC, employer contribution is 3 percent of CTC, employee deduction is a fixed 12,000.00 annually, and the allowance balances the remaining CTC.

Amount	Annual INR	Monthly INR
Basic	480,000.00	40,000.00
Employer contribution	36,000.00	3,000.00
Balancing allowance	684,000.00	57,000.00
Gross	1,164,000.00	97,000.00
Employee deduction	12,000.00	1,000.00
Net	1,152,000.00	96,000.00
CTC	1,200,000.00	100,000.00

Effective dates and concurrent edits

A new salary date must follow the latest version and cannot precede hire date. Future assignments are stored, but current insights and exports use the latest salary effective on or before the database current date. Assignment locks the active employee and rechecks template revision; template edits do not rewrite stored snapshots.

Annual and monthly periods are calculated independently. In rounding-sensitive examples, displayed monthly amounts multiplied by twelve can differ slightly from the annual breakdown.

People and salary API reference

Paths below are relative to **/api**. Feature JSON uses the shared response envelope. UI employee listing uses zero-based page indexes and caps page size at 100.

Method and path	Purpose
GET /health	Health probe; returns its own health shape
GET /countries	Supported country and local currency allowlist
GET /departments	Active seeded department references
GET /employees	Search, filter, sort, and page active employees
POST /employees	Validate and create a unique employee; HTTP 201
GET /employees/{id}	Read one employee profile
PUT /employees/{id}	Validate and edit an employee profile
DELETE /employees/{id}	Archive an employee; preserve salary and activity history
GET /employees/{id}/salary-history	Read dated salary versions newest first
POST /employees/{id}/salary-history	Direct append for integrations; HR UI uses template assignment
GET /salary-templates	Search and page templates; optional country and status
POST /salary-templates	Create a validated component template; HTTP 201
PUT /salary-templates/{id}	Revise using current revision; reject stale edits
POST /salary-assignments/preview	Calculate annual and monthly amounts without writing
POST /salary-assignments	Revalidate and append resolved salary snapshot; HTTP 201

Employee and template validation

Employees require number, first and last names, email, hire date, country, department, and job title. Services normalize identities and check uniqueness; database constraints remain the final guard. Template country determines its currency. Assignments require an active compatible template, current revision, positive CTC, and a valid effective date.

Insights files and API contracts

Method and path	Purpose
GET /insights/pay	Aggregate gross pay by country, department, or job title; separate currencies
GET /insights/outliers	IQR outliers by job title plus country plus currency; minimum five salaries
GET /imports/template	Download the employee salary workbook template
POST /imports/preview	Validate multipart workbook; return row and file errors without writes
POST /imports/confirm	Re-upload and revalidate; atomically create all rows or none; HTTP 201
GET /employees/export	Export active employees matching shared filters and sorting
GET /activity	Read newest-first paginated activity; optional action filter

Common response and status codes

```
{
  "status": 200,
  "success": true,
  "message": "Employees retrieved.",
  "data": { "items": [], "page": 0, "size": 25 },
  "errors": {}
}
```

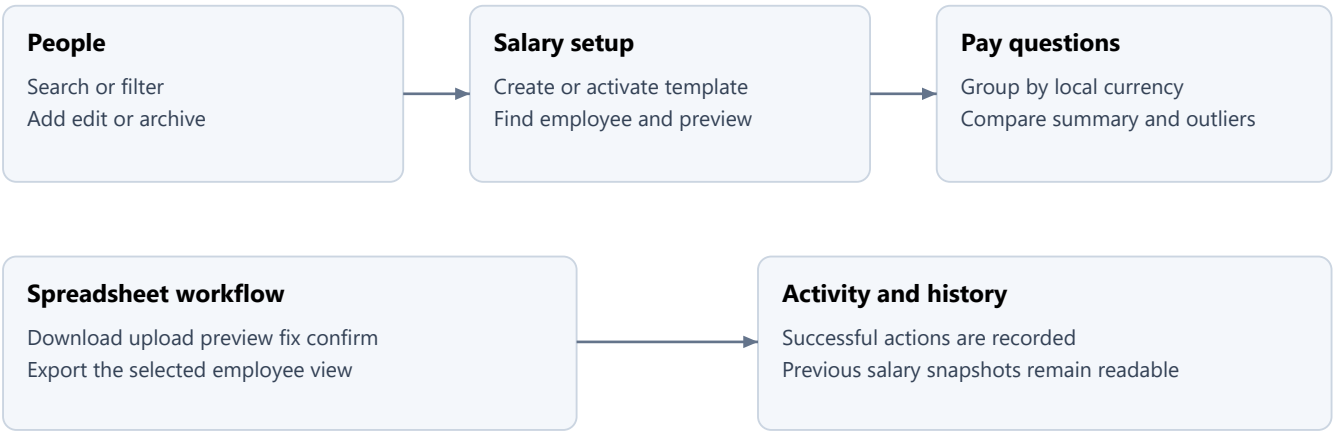
The envelope status matches the HTTP status. Successful creates and import confirmation use 201. Invalid or malformed requests use 400, missing records or routes 404, and uniqueness or stale-edit conflicts 409. Uploads above the multipart limit use 413. Unsupported methods or content types use 405 or 415. Unexpected failures return 500 with a generic message.

Field validation errors are keyed by field name and returned to the employee form. Binary workbook downloads and the health endpoint retain their own response shapes. No API returns JPA entities directly.

Pay insights

PostgreSQL calculates average, median, min, max, and range from current gross salaries. Currency is part of every aggregation group. Outlier bounds always use job title plus country plus currency, with at least five salaries. Values below Q1 minus 1.5 times IQR or above Q3 plus 1.5 times IQR are flagged. A flag is a review signal, not a conclusion about an employee.

User interface and module structure



The app shell uses a hamburger-controlled overlay side menu on desktop and mobile. It starts closed and closes on route selection, Escape, or backdrop click. Each workspace has short usage guidance, status labels where useful, and explicit loading, empty, error, or confirmation states.

Screen	Main workflow
/employees	Search and filter; create or edit a profile; archive; export view; direct salary shortcut
/employees/:id	Employee profile and immutable salary history
/salary-templates	Create, copy, revise, activate, or archive a country-specific template
/salary-assignments	Search employee; choose compatible active template; set annual CTC/date; preview and confirm
/insights	Group and filter pay summaries; review role outliers
/import-export	Download template; preview and fix rows; confirm import; filtered export
/activity	Read successful actions in newest-first order

Shared controls and API boundaries

TextFieldComponent and DropdownComponent share a plain 48px field shell with separate labels and accessible errors. FieldControl centralizes Angular Forms value, touched, disabled, and validation behavior. DropdownComponent uses Material Select for keyboard navigation and overlays. DropdownOptionsPipe maps reference lists; ServerPagedTableComponent owns pagination. Components call feature API services, which use the core HTTP service.

```
frontend/src/app/
  core/       shell API service interceptors notifications
  shared/     controls tables filters dialogs states helpers
  features/   employees salary insights import-export activity
```

Spreadsheets seeding and scale

Import contract

The workbook has thirteen required columns: Employee Number, First Name, Last Name, Work Email, Hire Date, Country Code, Department, Job Title, Currency, Basic, Allowances, Deductions, and Salary Effective Date.

Imports accept XLSX files up to 5 MB and 500 employee rows. Preview validates headers, required values, dates, supported references/currencies, identities, and salary rules; it writes nothing. Errors identify individual workbook rows. Confirmation uploads and validates the file again, rechecks database conflicts, and batch-inserts all valid employees and their first salary versions in one transaction. Existing employees are not updated.

Export behavior

Exports reuse active employee search/filter specifications and the sort allowlist. The service reads employee pages of 500 records, fetches current salaries for each page, and serializes a streaming XLSX workbook. Currency, gross, net, and effective date are included. Imported/manual first salaries do not carry template CTC provenance; template assignments do.

The database rows are fetched in bounded pages, but the final encoded workbook is returned as a byte array. For much larger future exports, a streaming response or background export job would be a separate improvement.

Deterministic seed

SeedDataGenerator produces exactly 10,000 fictional employees: five countries times ten job titles times 200 people. UUID v5 identifiers derive from stable employee numbers. Every group includes a clear high outlier. SeedDataRunner uses bounded batch inserts and conflict-safe IDs; rerunning does not duplicate the generated records.

The seed runs when APP_SEED_ENABLED is true. Docker Compose enables it automatically. This Java startup runner is the implemented seeding entry point; a separate shell seed script has not been added. The test reset adds a small fixed fixture set separately, so local test databases can have 10,000 ACME seed rows plus fixture employees.

Measured database performance

Local PostgreSQL 16.15 check	Warm cache time
25-row country-filtered employee page	1.819 ms
Country and currency pay aggregates	24.857 ms
Comparable-role outlier groups	29.732 ms

Measured on 14 October 2026 with 10,000 seeded rows plus six fixture employees. These are local measurements, not hosted guarantees. Query plans are retained in the performance report; no speculative cache or trigram extension was added.

Tests CI and AI collaboration

Evidence	Verified result at application commit 1dc488e
JUnit 5 and Mockito	33 passed; zero failures and errors
Angular TestBed and Vitest	17 passed across eight test files
Playwright sanity functional data	23 passed; zero failed or skipped
GitHub Actions	Backend, frontend, and browser-test jobs completed successfully

Open the verified GitHub Actions run. Backend coverage includes salary rules, import validation, references, activity, error envelopes, sorting, exports, deterministic seed, and UUID generation. Shared field tests cover form propagation, numeric values, required errors, model updates, disabled controls, and reference labels.

Browser setup and reports

The reusable TypeScript setup lives in e2e. Page objects cover employee list/detail, insights, import/export, and activity. Tests use stable test IDs and fixed fixtures. Functional fixtures reset before each applicable test; seed data invariants are checked separately through the API and PostgreSQL. Tests use Playwright waiting and assertions rather than random inputs or fixed sleeps.

```
npm run test:sanity
npm run test:functional
npm run test:data
npm run test:all
npm run report:open
npm run perf:check
```

BASE_URL and API_URL default to the local app; DB_URL enables direct database checks. Runs produce HTML, JUnit XML, and a short text summary. Screenshots are retained on failure; video and trace are captured on first retry. CI uploads e2e/reports, including query plans. Allure history and full tagged HR journey tests remain deferred SHOULD work.

Intentional use of Codex

The requirements and design were committed before feature work. Codex then worked in incremental slices with tests and review. The repository retains prompts, changes of direction, errors found, and corrections in ai. Human decisions set the scope, stack, currency rules, salary templates, and UI workflow. Generated code was reviewed and refined, including backend response contracts and shared frontend controls.

```
backend/.../salary/
  controller dto exception mapper model repository service
  specification validator util audit seed
Tests mirror production package paths.
```

Local setup deployment and handover

Run the complete local application

```
git clone https://github.com/rutupatel53/salary-management.git
cd salary-management
docker compose up --build
```

Reviewer access is required for the private repository. The UI opens at http://localhost:4200 and health at http://localhost:8080/api/health. Compose starts PostgreSQL 16, migrates the schema, starts Java 17, and seeds fictional data. The volume must belong to PostgreSQL 16; an older major-version data directory requires migration or a separate new volume.

Render and Netlify configuration

Render runs the backend Dockerfile. Connect the existing PostgreSQL database with DB_HOST, DB_PORT, DB_NAME, DB_USER, and DB_PASSWORD, or provision the paired service/database from render.yaml. Keep APP_SEED_ENABLED=true for the synthetic demo and APP_TEST_TOOLS_ENABLED=false. Never copy production credentials into source or documents.

Netlify builds frontend using netlify.toml and publishes dist/salary-management/browser. Its /api proxy must target the actual Render service URL. The remaining fallback sends client routes to index.html. Host GitHub integrations deploy on pushes; GitHub Actions contains no deploy job.

Public verification before submission

Confirm API health, a 10,000-row ACME seed count, employee search and edits, a template assignment, local-currency insights, invalid/valid import behavior, filtered export, and activity. Check browser errors, hosted cold start behavior, and the latest CI report. Attach the public app and demo links, grant reviewer access, and reply to the original assessment email.

Known limits and deliberate trade-offs

Departments and countries are currently fixed reference lists. Import is create-only and capped at 500 rows. There is no payroll, tax, payslip, approval, or role-based login. The configured assessment-hr-manager actor is labeled unverified. Templates use bounded JSONB instead of child component tables, and previews stay stateless. A modular monolith avoids infrastructure complexity for this assessment.

Public deployment verification, reviewer access, and the final submission remain handover tasks. The companion video demonstrates the local app with synthetic records; it should not be described as a hosted walkthrough until the public deployment has been verified.

Review entry points
https://github.com/rutupatel53/salary-management
docs/assessment-brief.md, docs/requirements.md, docs/design.md, docs/decisions.md, docs/performance.md, AGENTS.md, and ai/.